

TP Optimisation et Éléments Finis

0. Introduction

Exercice 1 : Révision de la méthode de Newton

1. Mettre en œuvre la méthode de Newton pour approcher le(s) zéro(s) de la fonction $f(x) = \exp(-x) - x$. Commentez.
2. Mettre en œuvre la méthode de Newton pour approcher le(s) zéro(s) de la fonction $f : x \mapsto x^2 - 2$. Commentez.
3. On cherche à approcher le(s) zéro(s) de la fonction $f : (x, y) \mapsto (\exp(x) - y, x^2 + y^2 - 16)$.
 - a) Donner une interprétation graphique du problème. En quoi cela aide-t-il ?
 - b) Mettre en œuvre la méthode de Newton. Commentez.

Exercice 2 : En dimension 1, révision... et un premier algorithme de recherche d'extrema

On se donne f une fonction définie sur $\mathbb{R}$ à valeurs dans $\mathbb{R}$.

1. Rappeler la formule de Taylor-Lagrange à l'ordre 2 au voisinage d'un point donné $x^* \in \mathbb{R}$.
On donnera les bonnes hypothèses de régularité à f pour cela.

On donne alors trois expressions de fonctions définies sur $\mathbb{R}$ et à valeurs dans $\mathbb{R}$:

- $f_1 : x \mapsto \frac{x}{x^2+1}$
- $f_2 : x \mapsto 2x^3 - 3x^2 - 12x + 4$
- $f_3 : x \mapsto x(-\cos(1) - \sin(1) + \sin(x))$

2. Pour chaque $i \in \{1, 2, 3\}$, répondre aux questions suivantes :
 - a. Représenter graphiquement f_i et sa dérivée f'_i .
 - b. La fonction f_i a-t-elle des minima et maxima locaux sur $\mathbb{R}$? sur $[-5, 5]$? Si oui, les indiquer approximativement à l'aide du graphe.
 - c. La fonction f_i possède-t-elle un maximum (resp. minimum) global sur $\mathbb{R}$? sur $[-5, 5]$? Si oui, l'indiquer sur le graphe.
 - d. Que dire des points où f'_i s'annule ? Donner un énoncé rigoureux de votre observation se basant sur la question 1.
 - e. Tracer f''_i . Quel est le signe de f''_i en les points de maxima et minima locaux ? Donner un énoncé rigoureux de votre observation.
2. À l'aide de la question 2. et de l'exercice 1., proposer une méthode numérique qui pourrait permettre de trouver les minima (ou maxima) locaux sur $\mathbb{R}$. La mettre en œuvre sur f_1 ou f_2 . Commentez.

Exercice 3 : Quelques formes quadratiques

On définit les fonctions suivantes de $\mathbb{R}^2$ à valeurs dans $\mathbb{R}$:

- $f_1 : (x, y) \mapsto x^2 + xy + 2y^2$
- $f_2 : (x, y) \mapsto x^2 + xy - 2y^2$
- $f_3 : (x, y) \mapsto 6x^2 + 3xy + 2y^2$
- $f_4 : (x, y) \mapsto -2x^2 + xy - \frac{7y^2}{2}$

1. Ces fonctions sont-elles des formes quadratiques ? Si oui, déterminer les matrices associées.
2. Pour chaque forme quadratique, déterminer les valeurs propres des matrices correspondantes.
3. Tracer la/les surfaces correspondantes pour ces mêmes fonctions ou quelques lignes de niveau. Ce que vous observez est-il en accord avec les résultats de la question 2 ? Expliquez.

Exercice 4 : En dimension 2

Soit g la fonction définie sur $\mathbb{R}^2$ et à valeurs dans $\mathbb{R}$ par $g : (x, y) \mapsto x^2 + y^2$.

1. Cette fonction admet-elle un minimum ? Si oui, en quel point et quelle est sa valeur ?

On s'intéresse à comprendre comment atteindre ce minimum en partant d'un point du plan.

2. Faire une représentation graphique de g .
2. Tracer la ligne de niveau correspondant à $g = 2$. Quelle est la nature géométrique de ce contour ? Tracer également d'autres contours (ex : $g = 1, g = 3$).
2. Calculer le gradient de g en un point $(x^*, y^*) \in \mathbb{R}^2$ donné. Comment est dirigé le gradient par rapport au plan tangent à la surface en ce point ?
2. Tracer sur un même graphique les lignes de niveau et quelques gradients en des points de chaque ligne de niveau. Commentez l'effet de suivre la direction et le sens de l'opposé du gradient de g par rapport à une direction quelconque.

1. Méthodes de gradients

Ex 1 : Méthode de gradient à pas constant

Pour chaque test, on pourra tracer l'évolution des itérées au fur et à mesure des itérations.

- 1) Programmer la méthode du gradient à pas constant.
- 2) Le tester sur la fonction $x \mapsto x^2 + \sin(x)$ définie sur $\mathbb{R}$.
- 3) Le tester également sur les fonctions $J : (x, y) \mapsto (x - 1)^2 + 10(y - 1)^2$ et $J : (x, y) \mapsto (x - 1)^2 + 10(x^2 - y)^2$ définies sur $\mathbb{R}^2$.
- 4) Le tester sur le problème de minimisation suivant : Trouver le minimum de la fonction $J : x \mapsto \sum_{i=1}^N \|x - A_i\|^2$ définie sur $\mathbb{R}^2$, où les $A_i, 1 \leq i \leq N$ sont des points donnés de $\mathbb{R}^2$.
On pourra par exemple tester avec $N = 4$.
- 5) Tester l'algorithme sur la fonction J de $\mathbb{R}^n$ dans $\mathbb{R}$ définie par l'expression suivante : Pour $x \in \mathbb{R}^n$,

$$J(x) = \sum_{i=1}^n i x_i^2$$

Ex 2 : Cas d'une fonctionnelle quadratique

1. Programmer la méthode de gradient à pas constant et la méthode de gradient à pas optimal pour le problème de minimisation de la fonctionnelle $J : x \mapsto \frac{1}{2}\langle Ax, x \rangle - \langle b, x \rangle$ à partir d'un vecteur initial x_0 avec A une matrice symétrique définie positive. Pour l'algorithme du gradient à pas optimal, on utilisera une expression explicite du pas optimal. Tester votre programme. Que dire de la vitesse de convergence?
2. Programmer et tester l'algorithme de gradient conjugué.
3. Comparer les deux approches. Qu'en pensez-vous?

2. Problèmes aux moindres carrés

Ex 3 : Quelques exemples de calcul

- 1) On se donne n points de $\mathbb{R}^2$, $(x_i, y_i)_{i \in \{1, \dots, n\}}$. Construire un programme qui donne la droite d'équation $y = ax + b$ qui minimise la quantité :

$$\sum_{i=1}^n |y_i - (ax_i + b)|^2$$

en utilisant la décomposition QR (on utilisera une fonction toute faite pour la décomposition QR). Prendre ensuite un exemple et faire une représentation graphique.

- 2) Recommencer avec une fonction quadratique à la place d'une fonction affine.

3. Gradient pas constant

Contexte

On se place dans $\mathbb{R}^2$ muni du produit scalaire euclidien canonique $\langle \cdot, \cdot \rangle$. On considère la fonctionnelle quadratique $J : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par :

$$J(x) = \frac{1}{2}\langle Ax, x \rangle - \langle b, x \rangle$$

avec une matrice symétrique $A = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ et un vecteur $b = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$.

L'objectif est de trouver le minimum de J sur $\mathbb{R}^2$.

Partie 1 : Questions Théoriques

1. **Gradient et Hessienne** : Calculer l'expression du gradient $\nabla J(x)$ et de la matrice Hessienne $\text{Hess}(J)(x)$ en tout point $x \in \mathbb{R}^2$.
2. **Convexité** : Justifier que la matrice A est définie positive. En déduire que la fonctionnelle J est α -convexe. Préciser la valeur de α en fonction des valeurs propres de A .
3. **Propriété du gradient** : Montrer que le gradient de J est M -lipschitzien, c'est-à-dire qu'il existe une constante $M > 0$ telle que $\|\nabla J(x) - \nabla J(y)\| \leq M\|x - y\|$ pour tout $(x, y) \in \mathbb{R}^2$. Que vaut M pour cette matrice A ?
4. **Convergence** : Énoncer la condition théorique sur le pas de descente $\rho > 0$ qui garantit la convergence de l'algorithme du gradient à pas constant pour cette fonctionnelle J .

Partie 2 : Implémentation et Analyse Numérique (Type TP)

Rappel de l'algorithme

L'algorithme du gradient à pas constant repose sur le principe de suivre, itération après itération, la direction de descente opposée au gradient. Il se formule de la manière suivante :

- **Initialisation** : On se donne un point de départ $x_0 \in \mathbb{R}^2$ et un pas de descente constant $\rho > 0$.
 - **Itération** : Pour $k \in \mathbb{N}$, on définit le point suivant par : $x_{k+1} = x_k - \rho \nabla J(x_k)$.
1. **Mise en équation** : En exploitant l'expression du gradient obtenue à la question 1, écrire explicitement la relation de récurrence vectorielle donnant x_{k+1} en fonction de x_k , de la matrice A , du vecteur b et du pas ρ .
 2. **Critères d'arrêt** : En pratique, l'algorithme ne peut pas tourner jusqu'à l'infini. Expliquez pourquoi il est judicieux de combiner un test d'arrêt basé sur un seuil de tolérance de la norme du gradient (par exemple $\|\nabla J(x_k)\| < \text{tol}$) et un critère de sécurité sur le nombre maximal d'itérations (`nitmax`).
 3. **Programmation** : Écrire le code (en Scilab ou Python) d'une fonction `gradient_pas_constant(A, b, rho, x0, tol, nitmax)` qui implémente cet algorithme. Votre fonction devra retourner la solution approchée x_{final} , ainsi que l'historique des itérées pour pouvoir les analyser.
 4. **Expérimentation numérique** : On fixe la condition initiale $x_0 = (0, 0)^T$, $\text{tol} = 10^{-6}$ et $\text{nitmax} = 1000$.
 - a) Tester votre algorithme avec un pas $\rho = 0.1$. L'algorithme converge-t-il ? Si oui, vers quelle solution ?
 - b) Tester ensuite l'algorithme avec un pas $\rho = 1$. Que se passe-t-il et comment l'expliquez-vous au regard de la théorie (question 4) ?
 5. **Visualisation** : Modifiez votre programme pour calculer la fonction coût $J(x_k)$ à chaque itération. Tracez l'évolution de la valeur $J(x_k)$ (ou de l'erreur $\|x_k - x^*\|$) en fonction du nombre d'itérations k . Que pouvez-vous déduire sur la vitesse de convergence de cette méthode ?

4. Gradient pas optimal

Contexte

On se place toujours dans $\mathbb{R}^2$ muni du produit scalaire euclidien canonique $\langle \cdot, \cdot \rangle$. On cherche à minimiser la fonctionnelle quadratique $J : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par :

$$J(x) = \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle$$

avec la matrice symétrique définie positive $A = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ et le vecteur $b = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$.

Contrairement à la méthode à pas constant, la méthode du gradient à pas optimal consiste à choisir à chaque itération k un pas ρ_k qui minimise la fonction $f_k(\rho) = J(x_k - \rho \nabla J(x_k))$ dans la direction de descente.

Partie 1 : Questions Théoriques

1. **Principe du pas optimal** : Écrire la condition nécessaire d'optimalité qui définit le pas ρ_k minimisant la fonction $\rho \mapsto J(x_k - \rho \nabla J(x_k))$.

2. **Orthogonalité des directions de descente** : En dérivant la fonction $f_k(\rho)$ par rapport à ρ , démontrer une propriété géométrique remarquable de cet algorithme : prouver qu'à chaque itération, les deux directions successives de descente $\nabla J(x_k)$ et $\nabla J(x_{k+1})$ sont orthogonales, c'est-à-dire $\langle \nabla J(x_{k+1}), \nabla J(x_k) \rangle = 0$.
3. **Calcul explicite du pas ρ_k** : Dans le cas spécifique de notre fonctionnelle quadratique, le pas optimal peut être calculé de manière exacte. En posant le vecteur résidu $r_k = b - Ax_k = -\nabla J(x_k)$, utiliser la relation d'orthogonalité démontrée à la question précédente pour prouver que le pas optimal est donné par la formule :

$$\rho_k = \frac{\langle r_k, r_k \rangle}{\langle Ar_k, r_k \rangle}$$

4. **Théorème de convergence** : Énoncer les hypothèses (régularité, convexité, propriété du gradient) du théorème garantissant la convergence de la méthode du gradient à pas optimal. Ces hypothèses sont-elles bien vérifiées pour notre matrice A ?

Partie 2 : Implémentation et Analyse Numérique (Type TP)

Rappel de l'algorithme

- **Initialisation** : On se donne un point de départ $x_0 \in \mathbb{R}^2$.
 - **Itération** : Pour $k \in \mathbb{N}$, on calcule le résidu $r_k = b - Ax_k$, on évalue le pas optimal $\rho_k = \frac{\langle r_k, r_k \rangle}{\langle Ar_k, r_k \rangle}$, puis on met à jour la position : $x_{k+1} = x_k + \rho_k r_k$.
1. **Programmation de l'algorithme** : Rédiger le code (en Scilab ou Python) d'une fonction `gradient_pas_optimal(A, b, x0, tol, nitmax)`. Remarquez que le pas ρ n'est plus un paramètre d'entrée puisqu'il est calculé dynamiquement à chaque itération.
 2. **Expérimentation numérique et comparaison** : On fixe $x_0 = (0, 0)^T$, $\text{tol} = 10^{-6}$ et $\text{nitmax} = 1000$. Comment le nombre d'itérations nécessaire pour atteindre la tolérance devrait-il se comparer à celui de la méthode à pas constant $\rho = 0.1$ de l'exercice précédent ?
 3. **Vérification numérique de la théorie** : Modifiez votre programme pour qu'il calcule et affiche à chaque itération le produit scalaire $\langle \nabla J(x_{k+1}), \nabla J(x_k) \rangle$. Quelle valeur numérique devriez-vous observer ?
 4. **Analyse de la vitesse de convergence** : L'inégalité de Kantorovitch stipule que l'erreur décroît avec un facteur lié au conditionnement de la matrice A , noté $\kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}$. Calculez les valeurs propres de A . L'algorithme convergera-t-il rapidement ou lentement sur ce cas précis ?

5. Gradient à pas constant avec projection

Contexte

On cherche à minimiser la fonctionnelle quadratique $J : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par :

$$J(x) = \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle$$

avec la matrice symétrique définie positive $A = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ et le vecteur $b = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$.

Cependant, on impose maintenant une contrainte : la solution doit appartenir à l'ensemble $K = \mathbb{R}_+^2$, c'est-à-dire le quart de plan où les coordonnées sont positives ou nulles ($K = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 \geq 0, x_2 \geq 0\}$). L'objectif est de trouver le minimum x^* de J sur K .

Partie 1 : Questions Théoriques

1. **Théorème de projection** : Rappeler la caractérisation géométrique de la projection $p_K(x)$ d'un point $x \in \mathbb{R}^2$ sur un sous-ensemble convexe, fermé et non vide K . Que peut-on dire de la régularité de l'application p_K ?
2. **Calcul de la projection** : L'ensemble $K = \mathbb{R}_+^2$ est un cas particulier de "pavé" (produit cartésien d'intervalles). Donner l'expression analytique explicite de la projection $p_K(x) = (p_K(x)_1, p_K(x)_2)$ pour un vecteur $x = (x_1, x_2) \in \mathbb{R}^2$.
3. **Point fixe de l'optimum** : En utilisant la caractérisation d'un minimum local sur un convexe et celle de la projection, démontrer que si $x^* \in K$ est le point de minimum de J sur K , alors pour tout pas $\rho > 0$, il vérifie l'équation de point fixe :

$$x^* = p_K(x^* - \rho \nabla J(x^*))$$

4. **Convergence** : Énoncer le théorème donnant la condition théorique sur le pas ρ qui garantit la convergence de l'algorithme du gradient avec projection vers la solution x^* .

Partie 2 : Implémentation et Analyse Numérique (Type TP)

Rappel de l'algorithme

Contrairement au cas sans contraintes, on doit s'assurer à chaque étape que l'on reste dans le domaine admissible K . L'algorithme se formule ainsi :

- **Initialisation** : On se donne un point de départ $x_0 \in K$ et un pas constant $\rho > 0$.
- **Itération** : Pour $k \in \mathbb{N}$, on calcule le point suivant en effectuant un pas de descente de gradient, puis en projetant le résultat sur K :

$$x_{k+1} = p_K(x_k - \rho \nabla J(x_k))$$

1. **Mise en équation vectorielle** : À l'aide de l'expression de la projection trouvée à la question 2, écrivez explicitement les formules de mise à jour itérative pour les composantes $x_{1,k+1}$ et $x_{2,k+1}$ du vecteur x_{k+1} .
2. **Programmation de l'algorithme** : Rédiger le code (en Scilab ou Python) d'une fonction `gradient_projeté_pas_constant(A, b, rho, x0, tol, nitmax)`. Attention au test d'arrêt : l'annulation du gradient ($\|\nabla J(x_k)\| < \text{tol}$) n'est plus un critère valide sous contraintes, car le minimum peut se trouver sur le bord du domaine (où le gradient n'est pas nul). Quel critère d'arrêt lié à l'évolution des itérées $\|x_{k+1} - x_k\|$ allez-vous implémenter ?
3. **Expérimentation et comparaison** : Fixez $x_0 = (0, 0)^T$ et $\rho = 0.1$. En traçant l'évolution des itérées (ou en l'analysant), que remarquez-vous sur le comportement de la coordonnée x_1 de la suite par rapport au minimum global non contraint des exercices précédents ? La projection a-t-elle été activée ?

Exercice 1 : Algorithme du gradient projeté

1. Programmer la méthode de gradient projeté à pas constant pour le problème de minimisation de la fonctionnelle $J : x \mapsto \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle$ sur l'ensemble $K = \mathbb{R}_+^d$, à partir d'un vecteur initial x_0 avec A une matrice symétrique définie positive et b un vecteur donné.

On pourra le programmer sous la forme d'une fonction qui prendra en argument deux variables `A` et `b`, une variable `rho` (le pas), une donnée initiale `u0` et les variables de contrôle des itérations `eps` et `nitmax`. La fonction retournera le dernier vecteur calculé.

2. Programmer la méthode de gradient projeté à pas constant pour le problème de minimisation de la fonctionnelle $J : x \mapsto \|Mx - f\|^2$ sur l'ensemble $K = \mathbb{R}_+^n$, à partir d'un vecteur initial x_0 avec M une matrice $n \times n$ inversible ($n \in \mathbb{N}^*$) et $f \in \mathbb{R}^n$.

Tester votre algorithme avec $M = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ et le vecteur $b = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$.

6. Méthode de pénalisation

Contexte

On cherche à minimiser la fonctionnelle quadratique $J : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par :

$$J(x) = \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle$$

avec la matrice symétrique définie positive $A = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ et le vecteur $b = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$.

On sait que le minimum global sans contrainte est $u_{\text{libre}} = (3/7, -6/7)^T$.

Cette fois-ci, on impose la contrainte d'inégalité $h(x) \leq 0$ avec $h(x) = -x_2$ (ce qui revient à imposer $x_2 \geq 0$). L'ensemble admissible est donc $K = \{x \in \mathbb{R}^2 \mid -x_2 \leq 0\}$. Comme u_{libre} ne vérifie pas cette contrainte, nous allons utiliser une méthode de pénalisation pour approcher la solution.

Partie 1 : Questions Théoriques

1. **Principe de la méthode** : Quel est le principe fondamental de la méthode de pénalisation par rapport à la gestion des contraintes géométriques ? Écrire la forme générale du problème d'optimisation pénalisé (P_ε) dépendant d'un paramètre $\varepsilon > 0$.
2. **Fonction de pénalisation** : Soit $\Psi : \mathbb{R}^2 \rightarrow \mathbb{R}$ la fonction de pénalisation. Quelles sont les propriétés mathématiques que Ψ doit vérifier vis-à-vis de l'ensemble K ? Proposez une expression explicite de $\Psi(x)$ pour notre contrainte d'inégalité $h(x) \leq 0$, en utilisant la fonction max.
3. **Théorème de convergence** : Énoncer le théorème de convergence garantissant que la solution du problème pénalisé, notée u_ε^* , converge vers la solution exacte u^* du problème sous contraintes lorsque $\varepsilon \rightarrow 0$. Préciser les hypothèses requises sur J et Ψ .

Partie 2 : Implémentation et Analyse Numérique (Type TP)

Rappel de l'approche

La méthode consiste à minimiser sur tout $\mathbb{R}^2$ (sans contrainte) la fonctionnelle pénalisée :

$$J_\varepsilon(x) = J(x) + \frac{1}{\varepsilon} \Psi(x)$$

avec $\Psi(x) = (\max(0, -x_2))^2$.

4. **Calcul du gradient pénalisé** : Montrer que la fonction Ψ est continûment différentiable (C^1) sur $\mathbb{R}^2$ et calculer l'expression du gradient de la fonctionnelle pénalisée $\nabla J_\varepsilon(x)$ en séparant les cas selon que la contrainte est respectée ($x_2 \geq 0$) ou violée ($x_2 < 0$).

5. **Programmation de l'algorithme** : Écrire le code (Scilab/Python) d'une fonction : `minimisation_penalisation(A, b, epsilon, x0, tol, nitmax)`. Dans cette fonction, vous devrez coder une simple boucle de descente de gradient (à pas constant) appliquée à la fonction $J_\varepsilon(x)$.
Note : Pour un ε fixé, l'algorithme s'arrête quand $\|\nabla J_\varepsilon(x_k)\| < \text{tol}$.
6. **Expérimentation et conditionnement** : En pratique lors du TP, vous allez tester l'algorithme avec des valeurs de pénalité de plus en plus fortes (par exemple $\varepsilon = 10^{-1}, 10^{-3}, 10^{-5}$). Quel problème numérique majeur allez-vous rencontrer lorsque ε devient très petit ?
7. Pour pallier ce défaut, on utilise souvent une méthode combinant pénalisation et multiplicateurs de Lagrange : comment s'appelle cette méthode alternative ?

7. Algorithme d'Uzawa

Contexte

On se place dans $\mathbb{R}^2$ et on cherche à minimiser la fonctionnelle quadratique $J : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par :

$$J(x) = \frac{1}{2} \langle Ax, x \rangle - \langle b, x \rangle$$

avec la matrice symétrique définie positive $A = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ et le vecteur $b = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$.

On introduit la contrainte d'inégalité $h(x) \leq 0$ avec $h(x) = x_1 + x_2 - 1$. L'ensemble admissible est donc $K = \{x \in \mathbb{R}^2 \mid h(x) \leq 0\}$. L'objectif est de trouver le minimum u^* de J sur K en utilisant l'algorithme d'Uzawa.

Partie 1 : Questions Théoriques

1. **Lagrangien** : Écrire l'expression du Lagrangien $L(x, \lambda)$ associé à ce problème de minimisation sous contrainte d'inégalité.
2. **Point selle et dualité** : Rappeler la définition d'un point selle (u^*, λ^*) pour le Lagrangien sur $\mathbb{R}^2 \times \mathbb{R}_+$. Expliquer brièvement pourquoi minimiser J sous contrainte revient à chercher un point selle du Lagrangien.
3. **Principe de l'algorithme** : L'algorithme d'Uzawa repose sur la recherche de ce point selle. Écrire les deux étapes itératives de cet algorithme (calcul de u^{k+1} puis de λ^{k+1}) à partir d'une initialisation $\lambda^0 \in \mathbb{R}_+$ et d'un paramètre $\mu > 0$. Comment peut-on interpréter l'étape de mise à jour du multiplicateur λ vis-à-vis du problème dual ?
4. **Théorème de convergence** : L'algorithme d'Uzawa est garanti de converger sous certaines conditions. Quelles sont les hypothèses requises sur J et h (convexité, régularité, caractère lipschitzien) ? Quelle est la condition stricte sur le paramètre μ (ou ρ) en fonction de la constante de forte convexité α de J et de la constante de Lipschitz de h ?

Partie 2 : Implémentation et Analyse Numérique (Type TP)

Rappel de l'approche

Contrairement à la pénalisation qui résout un problème approché, l'algorithme d'Uzawa calcule la solution exacte du problème contraint en alternant une minimisation sans contrainte sur x et une montée de gradient projeté sur le multiplicateur λ .

5. **Minimisation en u (Étape 1) :** À chaque itération k , on doit trouver u^k qui minimise la fonction $u \mapsto L(u, \lambda^k)$ sur $\mathbb{R}^2$. Pour notre problème spécifique où J est quadratique et h est affine, montrer que l'annulation du gradient de L par rapport à u conduit à résoudre le système linéaire suivant :

$$Au^k = b - \lambda^k c$$

où $c = (1, 1)^T$.

6. **Mise à jour de λ (Étape 2) :** En utilisant le fait que la projection d'un scalaire sur l'ensemble $\mathbb{R}_+$ correspond à la fonction $\max(0, \cdot)$, écrire explicitement la formule de récurrence calculant λ^{k+1} en fonction de λ^k, μ et $h(u^k)$.
7. **Programmation de l'algorithme :** Écrire le pseudo-code ou le code (Scilab/Python) d'une fonction `uzawa(A, b, c, d, mu, lambda0, tol, nitmax)` implémentant cette méthode (où c et d définissent la contrainte $h(x) = \langle c, x \rangle - d \leq 0$).
8. **Analyse du comportement :**
- Si à une itération k , le point u^k respecte largement la contrainte ($h(u^k) \ll 0$), que va-t-il se passer pour le multiplicateur λ^{k+1} lors des itérations suivantes ?
 - Quel est l'avantage fondamental de la méthode d'Uzawa sur la méthode de pénalisation quant au conditionnement du problème numérique lorsque l'on souhaite une grande précision ?

8. Algorithme Arrow-Hurwicz

Contexte

On se place dans $\mathbb{R}^2$ et on cherche à minimiser la fonctionnelle quadratique $J : \mathbb{R}^2 \rightarrow \mathbb{R}$ définie par :

$$J(u) = \frac{1}{2} \langle Au, u \rangle - \langle b, u \rangle$$

avec la matrice symétrique définie positive $A = \begin{pmatrix} 2 & 1 \\ 1 & 4 \end{pmatrix}$ et le vecteur $b = \begin{pmatrix} 0 \\ -3 \end{pmatrix}$.

On a la contrainte d'inégalité $\varphi(u) \leq 0$ avec $\varphi(u) = u_1 + u_2 - 1$. L'ensemble admissible est $C = \{u \in \mathbb{R}^2 \mid \varphi(u) \leq 0\}$.

Partie 1 : Questions Théoriques

- Comparaison avec Uzawa :** Rappeler les deux étapes de mise à jour itérative de l'algorithme d'Arrow-Hurwicz. Quelle est la différence fondamentale avec l'algorithme d'Uzawa concernant le traitement de la variable primale u à chaque itération ?
- Gradient du Lagrangien :** L'algorithme d'Arrow-Hurwicz ne cherche pas à annuler directement le gradient du Lagrangien par rapport à u , mais effectue une descente de gradient sur celui-ci. Donner l'expression analytique du vecteur directionnel $\nabla_u L(u, p) = \nabla J(u) + p \nabla \varphi(u)$ pour notre matrice A , notre vecteur b et la contrainte φ .
- Mise à jour duale :** L'étape de mise à jour du multiplicateur p s'écrit $p^{k+1} = \text{proj}_{\mathbb{R}_+}(p^k + \rho_2 \varphi(u^k))$. Justifiez que cette étape correspond toujours à une étape de montée de gradient projeté pour le problème dual consistant à maximiser le Lagrangien par rapport à p .

Partie 2 : Implémentation et Analyse Numérique (Type TP)

Rappel de l'algorithme

Contrairement à Uzawa qui minimise exactement en u , Arrow-Hurwicz utilise deux pas de descente distincts $\rho_1 > 0$ et $\rho_2 > 0$:

- **Initialisation** : On se donne $u^0 \in \mathbb{R}^2$ et $p^0 \in \mathbb{R}_+$.
- **Itération** : Pour $k \in \mathbb{N}$, on alterne :

$$\begin{aligned}u^{k+1} &= u^k - \rho_1 \left(\nabla J(u^k) + p^k \nabla \varphi(u^k) \right) \\p^{k+1} &= \text{proj}_{\mathbb{R}_+}(p^k + \rho_2 \varphi(u^k))\end{aligned}$$

4. **Mise en équation** : En posant le vecteur $c = (1, 1)^T$ et $d = 1$ de sorte que $\varphi(u) = \langle c, u \rangle - d$, remplacer les termes génériques de la définition de l'algorithme par les données de notre problème (A, b, c, d) pour obtenir les relations de récurrence explicites de u^{k+1} et p^{k+1} .
5. **Programmation de l'algorithme** : Rédiger le code (en Scilab ou Python) d'une fonction `arrow_hurwicz(A, b, c, d, rho1, rho2, u0, p0, tol, nitmax)`.
6. **Analyse du coût calculatoire** : En observant le système de la question 4, quel est l'avantage informatique majeur de l'algorithme d'Arrow-Hurwicz par rapport à la méthode d'Uzawa à chaque itération (que vous aviez mis en évidence lors de la résolution du système linéaire $Au^k = b - \lambda^k c$ pour Uzawa) ?
7. **Réglage empirique** : Lors du TP, vous allez constater que cet algorithme est parfois difficile à faire converger. Pourquoi la présence des deux paramètres ρ_1 et ρ_2 rend-elle la phase de réglage (tuning) de l'algorithme plus délicate en pratique ?